AUTOMATED SOFTWARE SUPPLY CHAIN SECURITY & SBOM AUDITOR

Comprehensive Architecture, Workflow Engine, and Source Code Technical Report

Project Name	Automated Software Supply Chain Security & SBOM Auditor
Project Type	Cybersecurity / Software Supply Chain Security Platform
Frontend Stack	React 18, TypeScript, Vite 5, React Router DOM v6, Tailwind CSS, Lucide Icons
Backend Stack	Python 3.14 / 3.13, FastAPI 0.115, Uvicorn, Pydantic v2, Structlog
Database / ORM	SQLite 3 (sbom_auditor_v2.db), SQLAlchemy 2.0 ORM
External Intelligence APIs	Google OSV.dev Batch Vulnerability API v1, npm Registry, PyPI API, Maven Central Solr API
Supported Ecosystems	npm (Node.js/JS/TS), PyPI (Python), Maven (Java/Kotlin/Gradle)
SBOM Specifications	CycloneDX v1.5 JSON, SPDX v2.3 JSON
Document Status	Complete Verified Technical Documentation
Date	August 2026

Executive Summary: This technical documentation provides an in-depth, verified breakdown of the Automated Software Supply Chain Security & SBOM Auditor codebase. It outlines the end-to-end security pipeline, directory hierarchy, core data models, REST API specifications, static manifest parsers, vulnerability scanning engines, risk scoring algorithms, and React frontend state architecture.

1. Project Overview

1.1 Non-Technical Overview (High-Level Purpose)

Modern software development relies heavily on open-source third-party libraries and packages. While these libraries accelerate development, they create a massive **Software Supply Chain** risk. When an open-source dependency contains a known vulnerability or malicious code, every application consuming that library becomes exposed to security exploits.

The **Automated Software Supply Chain Security & SBOM Auditor** acts as an automated digital security inspector. Users upload a zip archive of their project's codebase. The platform automatically scans the project, detects all open-source packages and sub-dependencies, creates a standardized **Software Bill of Materials (SBOM)**, checks public security databases for known zero-day vulnerabilities (CVEs and GHSAs), flags typosquatting or dangerous build scripts, and outputs an executive risk score alongside remediation steps.

Key Definitions & Core Concepts:

- **Software Supply Chain:** The combined ecosystem of third-party modules, build scripts, package managers, and upstream code required to assemble software applications.
- **SBOM (Software Bill of Materials):** A formal, structured inventory of all software components, libraries, versions, and relationships that make up an application (analogous to an ingredient list on packaged food).
- **Transitive Dependencies:** Packages that are not directly installed by the developer, but are required by third-party packages installed in the project (forming a deep dependency tree).
- **Typosquatting Attack:** Malicious packages published with names intentionally similar to popular packages (e.g., expreass instead of express) to trick developers into installing malware.
- **Lifecycle Script Risk:** Arbitrary code executed automatically during package installation (e.g., preinstall or postinstall hooks in npm) used by attackers to execute remote shell commands.

1.2 Technical Deep Dive (Architecture & Engine Capabilities)

Architecturally, the application is structured into a decoupled, high-performance web system consisting of a **React 18 SPA Frontend** and a **FastAPI Asynchronous Backend** with an integrated static analysis and security engine.

The platform addresses the limitations of manual dependency auditing through automated multi-ecosystem static analysis:

1. **Multi-Ecosystem Static Parsing:** Parses npm (package.json, package-lock.json, yarn.lock, pnpm-lock.yaml), PyPI (requirements.txt, pyproject.toml, setup.py, Pipfile.lock), and Maven/Gradle (pom.xml, build.gradle) without requiring full application execution.
2. **Asynchronous Batch Vulnerability Lookup:** Integrates with Google OSV.dev REST API via batch queries (sending up to 100 packages per HTTP POST) to minimize network latency and query CVE/GHSA vulnerability feeds.
3. **Heuristic Supply Chain Threat Detection:** Performs string distance calculations (Levenshtein distance) against a curated database of 1000+ top open-source libraries to flag typosquatting risks, and inspects package manifests for pre/post-install lifecycle scripts.
4. **Dynamic Weighted Risk Engine:** Calculates a normalized 0-100 overall project Risk Score using configurable weights for critical vulnerabilities, unpinned package versions, unknown/restrictive open-source licenses, and

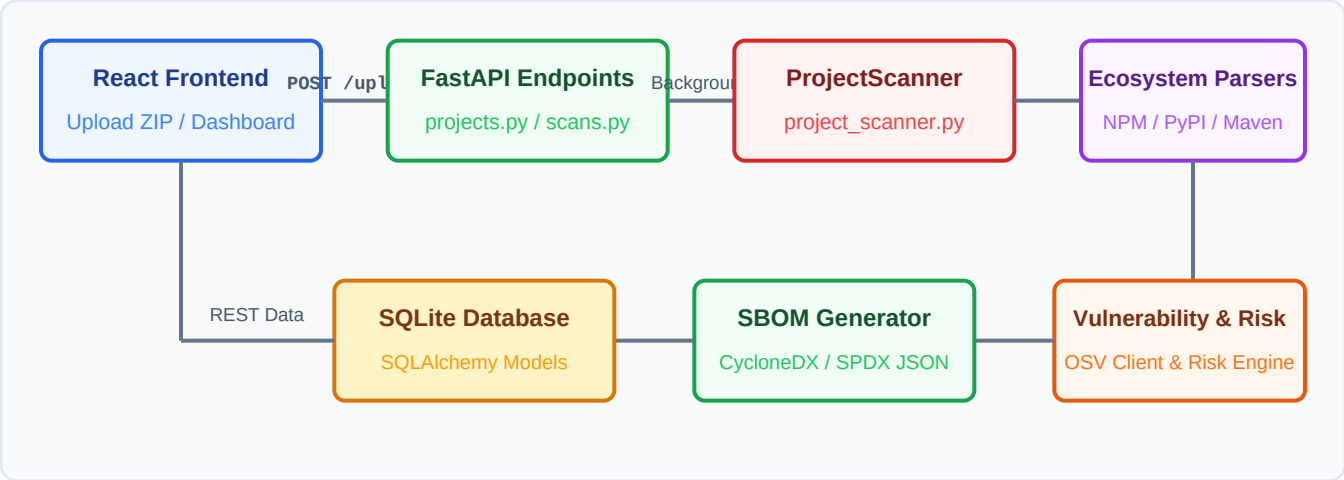
transitive risks.

5. **Standards-Compliant SBOM Exporter:** Generates downloadable JSON SBOMs compliant with international standards **CycloneDX v1.5** and **SPDX v2.3**.

2. Complete Project Workflow & Execution Engine

2.1 End-to-End System Workflow Execution Diagram

The diagram below illustrates the exact sequence of data flow and component interactions from user upload to dashboard rendering:



2.2 Detailed Step-by-Step Workflow Pipeline Specification

Step & File	Function / Handler	Input / Processing	Output Product
Step 1: Project Upload frontend/src/pages/ProjectDetail.tsx	`handleFileUpload()`	**In:** ZIP file selected by user **Process:** Sends FormData POST request to /api/projects/{id}/upload via api.ts Axios client.	**Out:** Scan object with 'pending' status
Step 2: File Security & Extraction backend/app/api/projects.py	`upload_and_scan()`	**In:** Multipart UploadFile (.zip) **Process:** Validates zip size (<50MB), path traversal attack check, safe temp extraction.	**Out:** Extracted project directory in ./temp_scans/
Step 3: Background Scan Task backend/app/api/projects.py	`run_scan_task()`	**In:** scan_id, zip_path **Process:** Launches FastAPI BackgroundTask executing project_scanner.scan_project().	**Out:** Scan status updated to 'running' in DB
Step 4: Ecosystem & Manifest Detection backend/app/services/manifest_detector.py	`ManifestDetector.detect()`	**In:** Extracted root directory **Process:** Walks directory tree, ignores node_modules/.venv, locates package.json, requirements.txt, pom.xml.	**Out:** List of detected manifest & lockfile paths
Step 5: Dependency Parsing & Tree Building backend/app/services/dependency_analyzer.py	`DependencyAnalyzer.analyze()`	**In:** Manifest file paths **Process:** Invokes NPMParser, PythonParser, or MavenParser. Resolves direct vs transitive packages.	**Out:** List of Dependency objects with versions and purls
Step 6: OSV Vulnerability Querying backend/app/services/vulnerability_scanner.py	`scan_dependencies()`	**In:** Dependencies list **Process:** Calls OSVClient.query_batch() with chunked payload (100 pkgs/batch) to hit https://api.osv.dev/v1.	**Out:** List of Vulnerability models (CVE/GHSA/CVSS)
Step 7: Registry Version Audit backend/app/services/version_checker.py	`VersionChecker.get_version_info()`	**In:** Package name & version **Process:** Fetches npmjs.org / PyPI API / Maven Central to check outdated status and patched versions.	**Out:** Latest & recommended package versions
Step 8: Supply Chain Threat Analysis backend/app/services/supply_chain_analyzer.py	`SupplyChainAnalyzer.analyze()`	**In:** Extracted files & deps **Process:** Runs TyposquattingDetector (Levenshtein check), LifecycleScriptAnalyzer (preinstall hooks), LicenseAnalyzer.	**Out:** List of RiskFinding models
Step 9: Dynamic Risk Engine Scoring backend/app/services/risk_engine.py	`RiskEngine.calculate_risk_score()`	**In:** Vulns & RiskFindings **Process:** Computes weighted risk score (0-100) and assigns risk level (CRITICAL, HIGH, MEDIUM, LOW).	**Out:** Normalized score & risk breakdown map
Step 10: SBOM Document Generation backend/app/services/sbom_generator.py	`SBOMGenerator.generate()`	**In:** Scan, Deps, Vulns **Process:** Formats standard CycloneDX v1.5 and SPDX v2.3 JSON documents.	**Out:** Serialized SBOM JSON string saved in DB
Step 11: Database Persistence backend/app/services/project_scanner.py	`ProjectScanner.scan_project()`	**In:** All analysis results **Process:** Commits dependencies, vulnerabilities, risk findings, and SBOMs to SQLite via SQLAlchemy.	**Out:** Scan status updated to 'completed'

Step 12: React Dashboard Update frontend/src/hooks/useApi.ts	`useProjects(), useLatestScan()`	**In:** REST API response **Process:** Axios fetches completed scan metrics; React components render tables, SVG trees, and charts.	**Out:** Updated Dashboard UI

3. Complete Verified Folder & Module Directory Structure

The directory tree below reflects the exact, verified file organization of the repository:

```
sbom/
■■■ README.md                          # Main project readme & execution guide
■■■ ARCHITECTURE.md                    # Architecture design document
■■■ RISK_SCORING.md                    # Risk Engine scoring mathematical model
■■■ SBOM.md                            # CycloneDX & SPDX spec overview
■■■ SECURITY.md                        # Security model & threat boundary
■■■ access.txt                         # Server manual startup instructions
■■■ backend/                           # FastAPI Python Backend Application
■   ■■■ run_server.ps1                 # Server startup script
■   ■■■ start_and_test.ps1             # Health test execution script
■   ■■■ test_scan.py                   # End-to-end integration test script
■   ■■■ test_server.py                 # Backend health test script
■   ■■■ requirements.txt               # Python package dependencies
■   ■■■ sbom_auditor_v2.db             # SQLite Production Database File
■   ■■■ app/                           # Core FastAPI Application Module
■       ■■■ main.py                    # Application factory, middleware & router registration
■       ■■■ config.py                  # Pydantic BaseSettings environment configuration
■       ■■■ api/                       # REST API Route Handlers
■       ■   ■■■ health.py              # Health check endpoints
■       ■   ■■■ projects.py            # Project CRUD & ZIP upload scan trigger
■       ■   ■■■ scans.py               # Scan metrics & dependency list endpoints
■       ■   ■■■ scans_2.py             # Dependency tree, risk breakdown, vulns & comparison
■       ■   ■■■ dependencies.py        # Dependency status table endpoints
■       ■   ■■■ vulnerabilities.py    # Vulnerability statistics & lookup endpoints
■       ■   ■■■ sbom.py                # CycloneDX/SPDX export & explorer endpoints
■       ■   ■■■ reports.py             # Compliance reports & summary JSON endpoints
■       ■■■ services/                  # Business Logic & Security Engines
■       ■   ■■■ project_scanner.py     # End-to-end scanner orchestrator
■       ■   ■■■ manifest_detector.py   # Manifest finder (package.json, requirements.txt, pom.xml)
■       ■   ■■■ dependency_analyzer.py # Multi-ecosystem analysis coordinator
■       ■   ■■■ npm_parser.py          # Node.js npm/yarn/pnpm parser
■       ■   ■■■ python_parser.py       # Python requirements/pyproject parser
■       ■   ■■■ maven_parser.py        # Java pom.xml/gradle parser
■       ■   ■■■ osv_client.py          # Google OSV.dev REST batch API client
■       ■   ■■■ vulnerability_scanner.py # Vulnerability detection orchestrator
■       ■   ■■■ version_checker.py     # Registry version comparison (npm/pypi/maven)
■       ■   ■■■ license_analyzer.py    # SPDX license risk classifier
■       ■   ■■■ typosquatting_detector.py # Levenshtein distance brandjacking detector
■       ■   ■■■ lifecycle_script_analyzer.py # Malicious install script inspector
■       ■   ■■■ supply_chain_analyzer.py # Integrated threat aggregator
■       ■   ■■■ risk_engine.py         # 0-100 dynamic risk score calculator
■       ■   ■■■ sbom_generator.py      # CycloneDX v1.5 & SPDX v2.3 generator
■       ■■■ models/                    # SQLAlchemy Database ORM Models
■       ■   ■■■ project.py             # Project & ProjectEcosystem tables
■       ■   ■■■ scan.py                # Scan table
■       ■   ■■■ dependency.py          # Dependency & DependencyRelationship tables
■       ■   ■■■ vulnerability.py       # Vulnerability table
■       ■   ■■■ risk_finding.py        # RiskFinding table
■       ■   ■■■ sbom.py                # SBOM table
■       ■   ■■■ license.py             # LicenseInfo & KnownLicense tables
■       ■■■ schemas/                   # Pydantic Validation & Serializer Schemas
■       ■   ■■■ project.py             # Project schemas
```

```
■       ■   ■■■ scan.py               # Scan schemas
■       ■   ■■■ dependency.py         # Dependency & tree node schemas
■       ■   ■■■ vulnerability.py      # Vulnerability schemas
■       ■   ■■■ risk.py               # Risk score & finding schemas
■       ■   ■■■ sbom.py               # SBOM explorer & export schemas
■       ■   ■■■ report.py             # Summary report schemas
■       ■■■ database/                 # Database Core
■       ■   ■■■ database.py           # SQLAlchemy engine, SessionLocal & get_db
■       ■■■ utils/                    # Security & Helper Utilities
■           ■■■ file_security.py      # Zip validation & traversal protection
■           ■■■ subprocess_security.py # Safe CLI execution helper
■           ■■■ logging.py            # Structlog JSON logging configuration
■           ■■■ version_utils.py      # Semver comparison utilities
■■■ frontend/                         # React + TypeScript + Vite SPA
    ■■■ package.json                  # Node dependencies & Vite scripts
    ■■■ vite.config.ts                # Vite server config & /api proxy to 8000
    ■■■ tsconfig.json                 # TypeScript compiler settings
    ■■■ tailwind.config.js            # Tailwind CSS design system tokens
    ■■■ index.html                    # HTML entry point
    ■■■ src/                          # React Application Source
        ■■■ main.tsx                  # React root mount point
        ■■■ App.tsx                   # React Router DOM route switchboard
        ■■■ layouts/                  # Layout Wrappers
        ■   ■■■ MainLayout.tsx        # Navigation sidebar & header wrapper
        ■■■ pages/                    # Primary Screen Views
        ■   ■■■ Dashboard.tsx         # Main security metrics & project cards
        ■   ■■■ Projects.tsx          # Project management & search
        ■   ■■■ ProjectDetail.tsx     # Single project scans & upload modal
        ■   ■■■ Vulnerabilities.tsx   # Vulnerability list & severity filters
        ■   ■■■ SBOMExplorer.tsx      # Interactive SBOM component search table
        ■   ■■■ DependencyTree.tsx    # Visual SVG dependency relationship graph
        ■   ■■■ Reports.tsx           # Compliance summary & export controls
        ■   ■■■ Settings.tsx          # System API timeouts & risk weights
        ■■■ hooks/                    # Custom React Hooks
        ■   ■■■ useApi.ts            # Stable useRef API data fetching hooks
        ■■■ services/                 # Network Services
        ■   ■■■ api.ts               # Axios HTTP client class
        ■■■ types/                    # TypeScript Interface Definitions
        ■   ■■■ index.ts             # Core entity type definitions
        ■■■ utils/                    # Formatting & Styling Helpers
            ■■■ helpers.ts            # Relative time, badge colors & cn class merger
```

4. Comprehensive File-by-File Technical Reference

The tables below provide a complete specification of every source file in the system.

File Path	Primary Purpose	Key Functions / Classes	Dependencies / Used By
backend/app/main.py	FastAPI application entry point, CORS middleware setup, lifespan initialization, router inclusion.	`app, lifespan, root()`	**Deps:** FastAPI, CORS, app.api **Used by:** Uvicorn ASGI Server
backend/app/config.py	Application environment settings management using Pydantic BaseSettings.	`Settings class`	**Deps:** pydantic-settings **Used by:** All backend modules
backend/app/database/database.py	SQLAlchemy engine setup, session maker, DB initialization, and dependency generator.	`get_db(), init_db()`	**Deps:** SQLAlchemy, app.config **Used by:** API route handlers
backend/app/api/projects.py	Project CRUD REST endpoints and async ZIP file upload trigger.	`create_project, list_projects, upload_and_scan`	**Deps:** FastAPI, ProjectScanner **Used by:** Frontend Projects/Dashboard pages
backend/app/api/scans.py	Endpoints for fetching scan summaries and package dependency lists.	`get_scan, get_scan_dependencies`	**Deps:** SQLAlchemy, Scan model **Used by:** Frontend ProjectDetail page
backend/app/api/scans_2.py	Endpoints for visual dependency tree, scan vulnerability listing, and risk breakdown.	`get_dependency_tree, get_scan_risk`	**Deps:** DependencyRelationship model **Used by:** Frontend DependencyTree/Dashboard pages
backend/app/api/dependencies.py	Endpoints for single dependency detail and scan status overview tables.	`get_dependency, get_dependency_status`	**Deps:** Dependency model **Used by:** Frontend Dependency Status table
backend/app/api/vulnerabilities.py	Vulnerability detail lookup and scan-level vulnerability severity statistics.	`get_vulnerability, get_vulnerability_stats`	**Deps:** Vulnerability model **Used by:** Frontend Vulnerabilities page
backend/app/api/sbom.py	CycloneDX/SPDX SBOM generation, download, and interactive explorer querying.	`get_sbom, download_sbom, explore_sbom`	**Deps:** SBOMGenerator **Used by:** Frontend SBOMExplorer page
backend/app/api/reports.py	Compliance summary reports and machine-readable JSON report exports.	`download_json_report, get_report_summary`	**Deps:** Scan/Risk models **Used by:** Frontend Reports page
backend/app/services/project_scanner.py	Main security audit orchestrator managing extraction, parsing, scanning, and persistence.	`ProjectScanner, scan_project()`	**Deps:** All parsers, scanners, ORM **Used by:** backend/app/api/projects.py

backend/app/services/manifest_detector.py	Scans directory tree for package manifest and lockfile paths across ecosystems.	`ManifestDetector, detect()`	**Deps:** pathlib, os.walk **Used by:** DependencyAnalyzer
backend/app/services/dependency_analyzer.py	Coordinates ecosystem-specific parsing and builds total package lists and stats.	`DependencyAnalyzer, analyze()`	**Deps:** NPMParser, PythonParser, MavenParser **Used by:** ProjectScanner
backend/app/services/npm_parser.py	Parses package.json, package-lock.json, yarn.lock, pnpm-lock.yaml for JS/TS projects.	`NPMParser, parse()`	**Deps:** json parsing, lockfile regex **Used by:** DependencyAnalyzer
backend/app/services/python_parser.py	Parses requirements.txt, pyproject.toml, setup.py, Pipfile.lock for Python projects.	`PythonParser, parse()`	**Deps:** tomllib, regex, semver **Used by:** DependencyAnalyzer
backend/app/services/maven_parser.py	Parses pom.xml and build.gradle files for Java/Kotlin projects.	`MavenParser, parse()`	**Deps:** xml.etree.ElementTree **Used by:** DependencyAnalyzer
backend/app/services/osv_client.py	Asynchronous HTTP client querying Google OSV.dev batch API with in-memory caching.	`OSVClient, query_batch()`	**Deps:** httpx, asyncio, cache **Used by:** VulnerabilityScanner
backend/app/services/vulnerability_scanner.py	Orchestrates vulnerability detection for resolved dependencies against OSV.dev.	`VulnerabilityScanner, scan_dependencies()`	**Deps:** OSVClient **Used by:** ProjectScanner
backend/app/services/version_checker.py	Queries npm, PyPI, and Maven registries to identify outdated packages.	`VersionChecker, check_dependencies()`	**Deps:** httpx, packaging.version **Used by:** ProjectScanner
backend/app/services/license_analyzer.py	Classifies open-source licenses into SPDX types and flags unknown or viral licenses.	`LicenseAnalyzer, analyze()`	**Deps:** SPDX mapping rules **Used by:** ProjectScanner
backend/app/services/typosquatting_detector.py	Detects brandjacking attacks via Levenshtein distance checks against 1000+ top libraries.	`TyposquattingDetector, check()`	**Deps:** Levenshtein calculation **Used by:** SupplyChainAnalyzer
backend/app/services/lifecycle_script_analyzer.py	Inspects package manifests for hazardous pre/post-install lifecycle scripts.	`LifecycleScriptAnalyzer, inspect()`	**Deps:** json parsing **Used by:** SupplyChainAnalyzer
backend/app/services/supply_chain_analyzer.py	Aggregates typosquatting, lifecycle scripts, unpinned dependencies, and license findings.	`SupplyChainAnalyzer, analyze()`	**Deps:** Typosquatting, Lifecycle, License **Used by:** ProjectScanner
backend/app/services/risk_engine.py	Calculates dynamic 0-100 overall project Risk Score using configurable weighted rules.	`RiskEngine, calculate_risk_score()`	**Deps:** app.config.settings **Used by:** ProjectScanner
backend/app/services/sbom_generator.py	Constructs standard CycloneDX v1.5 and SPDX v2.3 JSON documents.	`SBOMGenerator, generate()`	**Deps:** uuid, datetime **Used by:** ProjectScanner
backend/app/models/project.py	SQLAlchemy ORM definitions for projects and project_ecosystems tables.	`Project, ProjectEcosystem`	**Deps:** SQLAlchemy Base **Used by:** All API routes & services

backend/app/models/scan.py	SQLAlchemy ORM definition for scans table.	`Scan, ScanStatus`	**Deps:** SQLAlchemy Base **Used by:** ProjectScanner & API
backend/app/models/dependency.py	SQLAlchemy ORM definitions for dependencies and dependency_relationships tables.	`Dependency, DependencyRelationship`	**Deps:** SQLAlchemy Base **Used by:** Dependency services & API
backend/app/models/vulnerability.py	SQLAlchemy ORM definition for vulnerabilities table.	`Vulnerability, SeverityEnum`	**Deps:** SQLAlchemy Base **Used by:** VulnerabilityScanner & API
backend/app/models/risk_finding.py	SQLAlchemy ORM definition for risk_findings table.	`RiskFinding, RiskFindingType`	**Deps:** SQLAlchemy Base **Used by:** RiskEngine & API
backend/app/models/sbom.py	SQLAlchemy ORM definition for sboms table.	`SBOM`	**Deps:** SQLAlchemy Base **Used by:** SBOMGenerator & API
backend/app/schemas/*.py	Pydantic v2 schemas for request validation and response serialization.	`ProjectCreate, ScanResponse, etc.`	**Deps:** Pydantic v2 BaseModel **Used by:** FastAPI Route Handlers
backend/app/utils/file_security.py	Zip bomb protection, path traversal defenses, and safe extraction functions.	`validate_zip_file, safe_extract_zip`	**Deps:** zipfile, os, shutil **Used by:** backend/app/api/projects.py
frontend/src/App.tsx	React Router DOM route switchboard establishing layout routes.	`App()`	**Deps:** react-router-dom, MainLayout **Used by:** frontend/src/main.tsx
frontend/src/layouts/MainLayout.tsx	Primary application layout wrapper containing sidebar navigation and header search.	`MainLayout()`	**Deps:** react-router-dom, lucide-react **Used by:** frontend/src/App.tsx
frontend/src/pages/Dashboard.tsx	Main security operations center displaying project cards, scan metrics, and risk charts.	`Dashboard()`	**Deps:** useProjects, useLatestScan **Used by:** frontend/src/App.tsx
frontend/src/pages/Projects.tsx	Project management page with search, creation modal, and deletion controls.	`Projects()`	**Deps:** useProjects, api.createProject **Used by:** frontend/src/App.tsx
frontend/src/pages/ProjectDetail.tsx	Single project view with scan history table and new ZIP scan upload modal.	`ProjectDetail()`	**Deps:** useProject, useScans, api **Used by:** frontend/src/App.tsx
frontend/src/pages/Vulnerabilities.tsx	Vulnerability repository browser with severity badge filters and search.	`VulnerabilitiesPage()`	**Deps:** useScanVulnerabilities **Used by:** frontend/src/App.tsx
frontend/src/pages/SBOMExplorer.tsx	Interactive component inventory table with ecosystem/license/type filters.	`SBOMExplorerPage()`	**Deps:** useSBOMExplorer **Used by:** frontend/src/App.tsx
frontend/src/pages/DependencyTree.tsx	Visual SVG dependency tree renderer with risk-colored connection paths.	`DependencyTreePage()`	**Deps:** useDependencyTree **Used by:** frontend/src/App.tsx

frontend/src/pages/Reports.tsx	Compliance report overview, executive summary, and JSON download triggers.	`ReportsPage()`	**Deps:** useReportSummary, api **Used by:** frontend/src/App.tsx
frontend/src/pages/Settings.tsx	Configuration settings page for OSV timeouts, batch sizes, and risk scoring weights.	`SettingsPage()`	**Deps:** lucide-react **Used by:** frontend/src/App.tsx
frontend/src/services/api.ts	Axios HTTP client class wrapping REST calls to /api backend.	`ApiService class, api instance`	**Deps:** axios, frontend/src/types **Used by:** React custom hooks & pages
frontend/src/hooks/useApi.ts	Custom data-fetching hook with useRef stabilization to eliminate render loops.	`useApi(), useProjects(), etc.`	**Deps:** React useRef, useCallback **Used by:** All frontend pages
frontend/src/utils/helpers.ts	Utility helpers for formatting relative dates, CSS badge classes, and risk colors.	`formatRelativeTime, cn, etc.`	**Deps:** clsx, tailwind-merge **Used by:** All frontend components

5. Security, Risk Engine & Compliance Specifications

5.1 Risk Engine Mathematical Model

The project features a proprietary dynamic risk scoring engine (app/services/risk_engine.py). The engine computes a normalized overall Risk Score on a scale from 0.0 (Safe) to 100.0 (Critical Threat) using weighted penalty points:

Finding / Risk Category	Weight Penalty	Description & Impact Rule
Critical Vulnerability	`25.0 points / vuln`	Extremely severe vulnerability (CVSS >= 9.0 or Critical severity flag).
High Vulnerability	`15.0 points / vuln`	High severity vulnerability (CVSS 7.0 - 8.9).
Medium Vulnerability	`8.0 points / vuln`	Medium severity vulnerability (CVSS 4.0 - 6.9).
Low Vulnerability	`3.0 points / vuln`	Low severity vulnerability (CVSS 0.1 - 3.9).
Typosquatting Risk	`10.0 points / finding`	Package name Levenshtein distance <= 2 from top 1000 open source packages.
Lifecycle Script Risk	`5.0 points / script`	Presence of preinstall / postinstall build scripts in package manifest.
Transitive Vulnerability	`5.0 points / vuln`	Vulnerability residing deep in indirect dependency tree.
Unpinned Dependency	`4.0 points / dep`	Wildcard or unpinned version range (e.g. * or >=1.0.0).
Unknown / Restricted License	`3.0 points / dep`	Unrecognized license or restrictive copyleft license (GPL/AGPL).
Outdated Dependency	`2.0 points / dep`	Installed version is behind current registry latest release.

Normalized Risk Level Thresholds:

- **CRITICAL:** Score >= 75.0 (Requires immediate build halt & patch deployment)

- **HIGH:** Score 50.0 - 74.9 (High-priority security updates needed)
- **MEDIUM:** Score 25.0 - 49.9 (Moderate risk; schedule updates)
- **LOW:** Score 0.0 - 24.9 (Safe build posture; routine maintenance)